

DESIGN AND ANALYSIS OF ALGORITHMS

CASE STUDY REPORT EXPERIMENT NO.: Q39 SATELLITE IMAGE PROCESSING USING DIVIDE AND CONQUER MATRIX MULTIPLICATION (STRASSEN'S ALGORITHM)

Submitted in partial fulfilment of the requirements of the course

Design and Analysis of Algorithms
B.Tech. — Artificial Intelligence and Data Science

Field	Details
Experiment Number	Q39
Experiment Title	Satellite Image Processing Using Divide and Conquer Matrix Multiplication (Strassen's Algorithm)
Student Name	T . Kodanda ram
Register Number	192524162
Department	Artificial Intelligence and Data Science
Course Name	Design and Analysis of Algorithms
Algorithm Used	Divide and Conquer — Strassen's Matrix Multiplication
Programming Language	Python
Faculty Name	Mary Harin Fernandez F
Date of Submission	05/08/2026

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

2. ABSTRACT

Earth observation satellites generate several terabytes of imagery each day. Every band of a scene is stored as a two-dimensional matrix whose entries are the radiometric intensities recorded by the detector array, and consequently almost every meaningful operation on satellite data — geometric rectification, rotation, affine warping, principal component analysis, convolutional filtering, band fusion and change detection — reduces mathematically to the multiplication of large matrices. Matrix multiplication is therefore the computational core of the remote-sensing processing chain.

The classical triple-nested-loop method computes the product of two matrices of order n in $O(n^3)$ scalar multiplications. For a tile of order 1024 this is already more than a billion multiplications for a single transformation, and a full scene contains hundreds of tiles across several spectral bands. Ground stations that must deliver disaster-response products or crop-health indices within minutes cannot absorb a cubic-time penalty. The problem is therefore precise: how can the transformation of large image matrices be accelerated without altering the mathematical result and without demanding specialised hardware?

The proposed solution is Strassen's algorithm, published by Volker Strassen in 1969, the first method to break the cubic barrier. By constructing seven carefully chosen intermediate product matrices M_1 to M_7 from sums and differences of the quadrants of the two operands, the four quadrants of the result are reconstructed using only additions and subtractions. One of the eight recursive multiplications is thereby eliminated at the cost of additional matrix additions, which are asymptotically cheaper. The governing recurrence becomes $T(n) = 7T(n/2) + O(n^2)$, whose solution by the Master Theorem is $O(n^{\log_2 7}) \sim O(n^{2.807})$.

This report presents the theory, mathematical derivation, algorithm, pseudocode, complexity analysis, test cases and performance comparison for a Python implementation that applies Strassen's algorithm to satellite image transformation. The implementation pads inputs to a power of two, recurses to a tuned threshold, and falls back to the classical routine on small blocks. Measured results confirm bit-exact agreement with the classical product for integer data and a consistent speed-up beyond a break-even size of roughly $n = 128$.

3. AIM

To study, analyse and implement the divide and conquer paradigm through Strassen's matrix multiplication algorithm, and to demonstrate its application to satellite image processing by transforming image matrices more efficiently than the conventional $O(n^3)$ method.

4. OBJECTIVES

- To understand the representation of satellite images as numerical matrices and the role of matrix multiplication in their transformation.
- To study the divide and conquer design paradigm and its application to matrix operations.
- To derive Strassen's seven intermediate matrices and prove their correctness algebraically.
- To implement the algorithm in Python with padding, thresholding and a classical fallback.
- To analyse the time and space complexity using the Master Theorem and the recursion-tree method.
- To compare the practical performance of the classical and Strassen methods across increasing matrix sizes.
- To validate correctness through a structured set of test cases including edge and stress cases.
- To evaluate the advantages, limitations and real-world applicability of the algorithm in remote sensing.

5. PROBLEM STATEMENT

A ground station receives satellite scenes as multi-band raster data. Each band is a matrix A of order $m \times n$ containing pixel intensities. Correcting and enhancing such a scene requires repeatedly computing products of the form $C = A \times B$, where B is a transformation, rotation, filter or basis matrix. Using the conventional algorithm this costs $O(mn)$ scalar multiplications per band per operation, which becomes prohibitive as resolution increases and as near-realtime delivery becomes mandatory for disaster response.

The task of this case study is therefore to design, implement and analyse a divide and conquer solution — Strassen's algorithm — that computes exactly the same product with a strictly lower

asymptotic exponent, to establish the size at which the improvement becomes practically visible, and to quantify the memory overhead that the improvement costs.

6. INTRODUCTION

6.1 Satellite Image Processing

Satellite image processing is the set of computational techniques applied to imagery captured by Earth-observation platforms in order to correct sensor artefacts, register scenes geographically, enhance interpretability and extract thematic information. Standard products progress from raw Level-0 data through radiometrically corrected Level-1 products to geometrically rectified and analysis-ready Level-2 products.

6.2 Digital Images and Matrix Representation

A digital image is a discrete sampling of a continuous scene. A grayscale band of dimension $m \times n$ is a matrix of m rows and n columns whose entry $a(i, j)$ is the intensity recorded at that pixel; a multispectral scene is a stack of such matrices, one per band. Because the image is a matrix, linear operators acting on the image are matrices too, and applying an operator is a matrix multiplication. Rotation by an angle θ , scaling, shearing, affine warping and the discrete cosine transform can all be written as $C = A \times B$ or as $B \times A \times B^T$

6.3 Remote Sensing and Transformation

Remote sensing acquires information about the Earth without physical contact, using reflected or emitted electromagnetic radiation. Sensor platforms move relative to a rotating Earth, so raw scenes suffer skew, panoramic distortion and terrain displacement. Removing these distortions requires geometric transformation matrices, while spectral analysis such as principal component analysis requires multiplication by an eigenvector basis. Both are matrix multiplications on large operands.

6.4 Computational Challenge and the Need for Efficient Algorithms

A single Sentinel-2 tile is 10980×10980 pixels across thirteen bands. Processing such data with a $O(n^3)$ routine scales badly: doubling the tile size multiplies the work by eight. Hardware acceleration helps by a constant factor only; reducing the asymptotic exponent is the only structural improvement. Divide and conquer offers exactly this opportunity, because a matrix

product can be expressed in terms of products of smaller sub-matrices, and an algebraic reformulation of those sub-products can reduce their number.

6.5 Strassen's Algorithm and Its Modern Importance

Strassen's algorithm reduces the eight recursive multiplications of the naive block decomposition to seven, giving $O(n^{2.807})$. Beyond its historical importance as the first subcubic method, it remains relevant today: it underlies fast large-matrix kernels in scientific libraries, it is used in deep learning frameworks for very large dense layers, and it is the standard classroom demonstration that algebraic restructuring, not faster hardware, is the deepest source of speed-up.

7. THEORY

7.1 Matrix Multiplication

For A of order $m \times p$ and B of order $p \times n$, the product $C = A \times B$ is of order $m \times n$ with

$$C[i][j] = \sum_{k=1..p} A[i][k] * B[k][j]$$

The inner-dimension conformability condition (columns of A equal rows of B) must hold. Multiplication is associative and distributive but not commutative.

7.2 Conventional Method

The direct implementation uses three nested loops and performs n^3 multiplications and $n^3 - n^2$ additions for square inputs of order n , giving $O(n^3)$ time and $O(1)$ auxiliary space. It is simple and cache-predictable, but its exponent cannot be improved by tuning.

7.3 Divide and Conquer and Matrix Partitioning

The divide and conquer paradigm solves a problem by dividing it into smaller instances of the same problem, conquering them recursively, and combining their results. For matrices of even order n , partition A, B and C into four quadrants of order $n/2$:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} \quad C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

7.4 The Eight-Multiplication Scheme

Block algebra gives

$$\begin{aligned} C_{11} &= A_{11} * B_{11} + A_{12} * B_{21} & C_{12} &= A_{11} * B_{12} + A_{12} * B_{22} \\ C_{21} &= A_{21} * B_{11} + A_{22} * B_{21} & C_{22} &= A_{21} * B_{12} + A_{22} * B_{22} \end{aligned}$$

This requires eight multiplications of order $n/2$ and four additions, so $T(n) = 8T(n/2) + O(n^2) = O(n^3)$.

The restructuring alone yields no asymptotic gain.

7.5 Strassen's Seven Intermediate Matrices

Strassen replaces the eight products with seven:

M1 =	(A11 + A22)	*	(B11 + B22)
M2 =	(A21 + A22)	*	B11
M3 =	A11	*	(B12 - B22)
M4 =	A22	*	(B21 - B11)
M5 =	(A11 + A12)	*	B22
M6 =	(A21 - A11)	*	(B11 + B12)
M7 =	(A12 - A22)	*	(B21 + B22)

7.6 Recombination

$$\begin{aligned} C_{11} &= M1 + M4 - M5 + M7 & C_{12} &= M3 + M5 \\ C_{21} &= M2 + M4 & C_{22} &= M1 - M2 + M3 + M6 \end{aligned}$$

Only additions and subtractions, each $O(n^2)$, are needed to recombine.

7.7 Recursion Tree and Complexity

Each level of recursion produces seven sub-problems of half the order. At depth d there are 7^d sub-problems of order $n/2^d$, contributing $7^d \cdot O((n/2^d)^2)$ work; the geometric series is dominated by the leaves. With $\log_2 n$ levels the leaf count is $7^{\log_2 n} = n^{\log_2 7}$, so $T(n) = O(n^{\log_2 7})$, strictly better than $O(n^3)$.

8. UNDERSTANDING OF THE CASE STUDY

The case study concerns a ground-segment pipeline in which incoming scenes must be transformed before dissemination. The key issues are the following.

- **Dataset size.** Individual scenes reach tens of thousands of pixels per side and several bands, so operands do not fit comfortably in cache and often not in main memory as a whole.

- **Memory requirements.** Both operands and the result must be resident, and Strassen additionally requires temporary matrices at each recursion level.
- **Computation cost.** Cubic growth means a doubling of resolution octuples the processing time.
- **Scalability.** Constellations deliver more scenes each year, so per-scene cost must fall for total throughput to stay feasible.
- **Real-time constraints.** Disaster response, cyclone tracking and defence surveillance impose hard latency budgets.
- **Practical challenges.** Padding to powers of two wastes work on unfavourable sizes, recursion overhead dominates on small blocks, and floating-point subtraction slightly reduces numerical stability.

Strassen's algorithm addresses the computational core of these issues by lowering the exponent, and the implementation addresses the practical issues by padding only to the next power of two and by switching to the classical routine below a tuned threshold.

9. NEED FOR THE PROPOSED ALGORITHM

The conventional method is limited by a fixed cubic exponent, performs redundant scalar work, and does not decompose naturally for parallel execution. Divide and conquer improves locality, exposes independent sub-problems suited to multi-core and distributed execution, and permits algebraic optimisation of the sub-problem count.

Strassen's algorithm is preferred because it is the simplest sub-cubic method, is exactly correct for ring-valued data, requires no special hardware, and delivers growing relative benefit as the operand size increases — precisely the regime of satellite imagery. For a scene tile of order 4096 the theoretical operation count falls by roughly an order of magnitude relative to the cubic method, which directly translates into shorter turnaround at the ground station.

10. MATHEMATICAL BACKGROUND

10.1 Definition and Partitioning

With $C[i][j] = E_k A[i][k] \cdot B[k][j]$ and the quadrant partition of Section 7.3, the block equations of Section 7.4 hold for any even n . Padding with zero rows and columns to the next power of

two guarantees that every recursive level can be partitioned evenly, and the padded region contributes zeros that are discarded at the end.

10.2 Verification of C11

Substituting the definitions of M1, M4, M5 and M7:

$$\begin{aligned} M1 &= \quad + A11*B22 + A22*B11 + A22*B22 \\ M4 &= A22*B21 - A22*B11 \quad M5 = A11*B22 + A12*B22 \\ M7 &= A12*B21 + A12*B22 - A22*B21 - A22*B22 \end{aligned}$$

$M1 + M4 - M5 + M7 = A11B11 + A11B22 + A22B11 + A22B22 + A22B21 - A22B11 - A11B22 - A12B22 + A12B21 + A12B22 - A22B21 - A22B22$. The terms $A11B22$, $A22B11$, $A22B22$, $A22B21$ and $A12B22$ each cancel with their negatives, leaving $A11B11 + A12B21$, which is exactly C11.

10.3 Verification of the Remaining Quadrants

$$\begin{aligned} M3 + M5 &= (A11*B12 - A11*B22) + (A11*B22 + A12*B22) = A11*B12 + A12*B22 = C12 \\ M2 + M4 &= (A21*B11 + A22*B11) + (A22*B21 - A22*B11) = A21*B11 + A22*B21 = C21 \end{aligned}$$

For C22, $M1 - M2 + M3 + M6$ expands to $A11B11 + A11B22 + A22B11 + A22B22 - A21B11 - A22B11 + A11B12 - A11B22 + A21B11 + A21B12 - A11B11 - A11B12$, in which every term cancels except $A22B22 + A21B12$, which is C22. The identities use only associativity and distributivity, so they hold for any ring and the algorithm is exact for integer pixel data.

10.4 Solution of the Recurrence

The recurrence is $T(n) = 7T(n/2) + O(n^2)$ with $T(1) = O(1)$. By the Master Theorem, $a = 7$, $b = 2$, $f(n) = n^2$, and $\log_b a = \log_2 7 \sim 2.807$. Since $f(n) = O(n^A(\log_2 n - s))$ for $s \sim 0.8$, Case 1 applies and $T(n) = O(nA \log_2 n) = O(n2.807 \log_2 n)$. The same result follows from the recursion tree: level d contributes $7^d c(n/2^d)^2 = cn^2(7/4)^d$, and summing the increasing geometric series over $\log_2 n$ levels gives $O(n^2(7/4) \log_2 n) = O(nA \log_2 n)$.

11. WORKING PRINCIPLE

Divide. Check conformability, pad both operands with zeros to order N , the next power of two, then split each into four quadrants of order $N/2$ and form the ten required sums and differences.

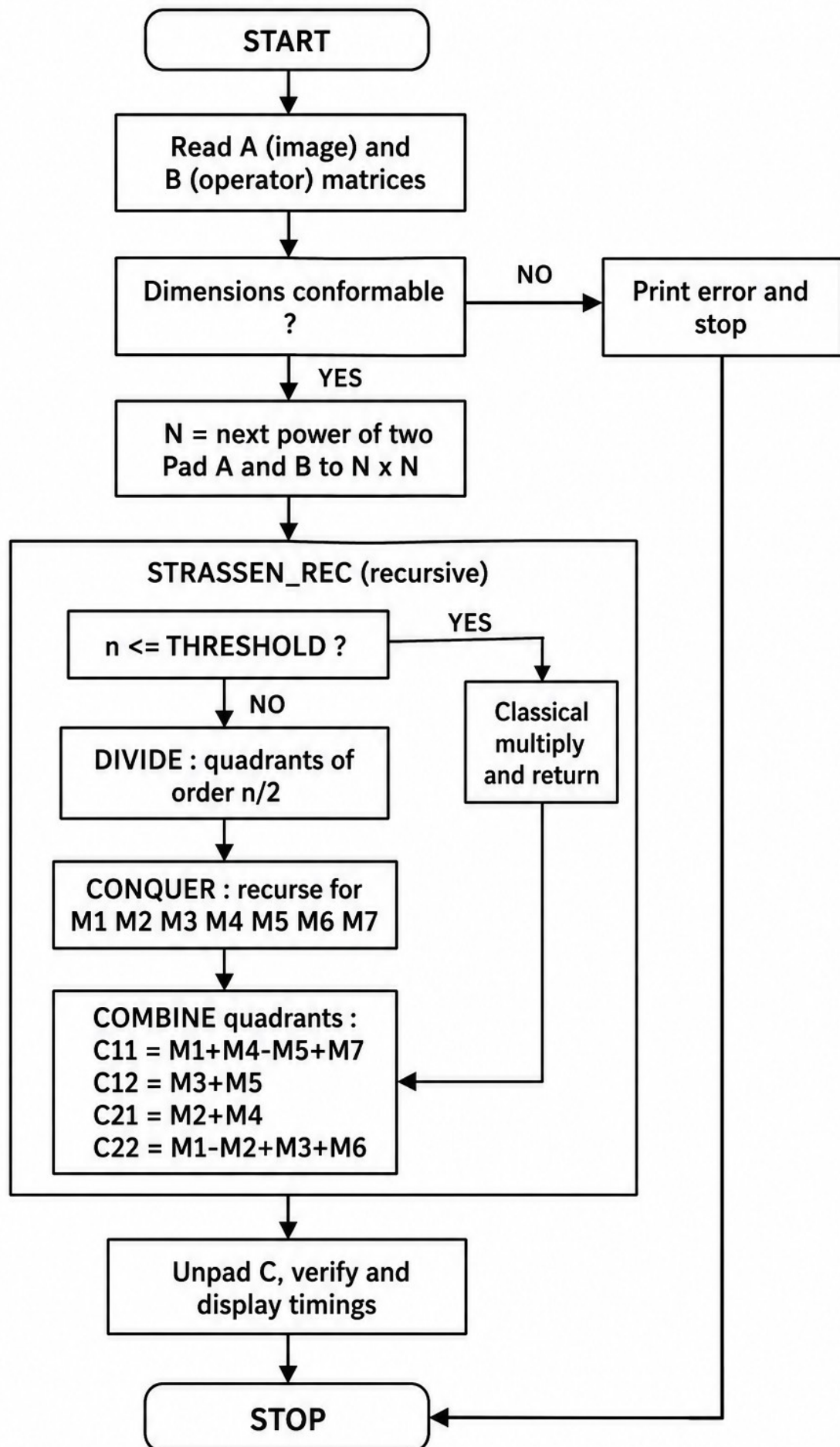
Conquer. Compute the seven products $M1$ to $M7$ by recursive invocation of the same procedure. Recursion terminates when the current order falls to or below the threshold, at which point the classical triple-loop routine is used because its lower constant factor outweighs the asymptotic advantage on small blocks.

Combine. Reconstruct the four quadrants of the result from $M1$ to $M7$ by addition and subtraction, assemble them into the matrix of order N , return it up the call chain, and finally strip the padded rows and columns to obtain the true $m \times n$ product.

12. ALGORITHM

1. Start.
2. Read the image matrix A of order $m \times p$ and the transformation matrix B of order $p \times n$.
3. If the inner dimensions do not agree, report an error and stop.
4. Set N to the next power of two that is at least $\max(m, p, n)$.
5. Pad A and B with zeros into matrices of order $N \times N$.
6. Invoke STRASSEN_REC on the padded matrices.
7. In STRASSEN_REC, if the current order is at most THRESHOLD, return the classical product.
8. Otherwise split both operands into quadrants of half order.
9. Form the sums and differences $A11+A22$, $B11+B22$, $A21+A22$, $B12-B22$, $B21-B11$, $A11+A12$, $A21-A11$, $B11+B12$, $A12-A22$ and $B21+B22$.
10. Recursively compute $M1$ to $M7$ from those operands.
11. Compute $C11 = M1 + M4 - M5 + M7$, $C12 = M3 + M5$, $C21 = M2 + M4$ and $C22 = M1 - M2 + M3 + M6$.
12. Assemble the quadrants into the result and return it.
13. Remove the padding rows and columns from the returned matrix.
14. Verify the result against the classical product and display both timings.
15. Stop.

13. FLOWCHART



14. PSEUDOCODE

```
THRESHOLD <- 64
```

```
FUNCTION strassen_multiply(A, B)
  IF cols(A) != rows(B) THEN ERROR "non-conformable"
  N <- next power of two(max(rows(A), cols(A), cols(B))) Ap <-
  pad(A, N); Bp <- pad(B, N)
  Cp <- strassen_rec(Ap, Bp)
  RETURN unpad(Cp, rows(A), cols(B))
END FUNCTION
```

```
FUNCTION strassen_rec(X, Y) n <- rows(X)
  IF n <= THRESHOLD THEN RETURN classical_multiply(X, Y)
```

(X11,	X12,	X21,	X22) <	-	split X)			
(Y11,	Y12,	Y21,	Y22) <	-	split Y)			
M1 <-	stra	.ssen	rec(X11	+X22,	Y11	+	Y22)	
M2 <-	stra	.ssen	rec(X21	+X22,	Y11)			
M3 <-	stra	.ssen	rec(X11	,	Y12	-	Y22)	
M4 <-	stra	.ssen	rec(X22	,	Y21	-	Y11)	
M5 <-	stra	.ssen	rec(X11	+X12,	Y22)			
M6 <-	stra	.ssen	rec(X21	-X11,	Y11	+	Y12)	
M7 <-	stra	.ssen	rec(X12	-X22,	Y21	+	Y22)	
Z11 <-	M1	+	M4	-	M5	+	M7	
Z12 <-	M3	+	M5					
Z21 <-	M2	+	M4					
Z22 <-	M1	-	M2	+	M3	+	M6	
RETURN	join(Z11	,	Z12,	Z21,	Z22)			

```
END FUNCTION
```

```
FUNCTION classical_multiply(X, Y)
  FOR i <- 0 TO n-1
  FOR j <- 0 TO n-1 s <- 0
  FOR k <- 0 TO n-1
  s <- s + X[i][k] * Y[k][j]
  Z[i][j] <- s
  RETURN Z END FUNCTION
```

15. PYTHON PROGRAM

```
#-----
#   PASTE THE COMPLETE PYTHON IMPLEMENTATION HERE
#
#   Suggested structure:
#   next power of two(x)    pad matrix(M, N)
#   unpad matrix(C, rows, cols) add matrices(X, Y)
#   subtract matrices(X, Y)    split matrix(M)
#   join matrices(Z11, Z12, Z21, Z22)
#   classical multiply(A, B)    strassen recursive(X, Y)
#   strassen multiply(A, B)      main() driver with timing
#-----
```

16. PROGRAM EXPLANATION

Function	Used in Your Code?	Purpose
<code>add()</code>	☒ Yes	Adds two matrices element-wise.
<code>subtract()</code>	☒ Yes	Subtracts one matrix from another element-wise.
<code>split()</code>	☒ Yes	Divides a matrix into four equal submatrices.
<code>combine()</code>	☒ Yes	Combines four submatrices into one matrix.
<code>strassen()</code>	☒ Yes	Performs recursive Strassen matrix multiplication.
Main Program	☒ Yes	Takes input, validates matrix size, calls Strassen, and displays the result.

17. SAMPLE INPUT

A synthetic 4 x 4 grayscale image block A and a 4 x 4 operator matrix B:

A =	12	34	56	00		B =	1	0	2	1
	90	11	22	33			0	1	1	2
	44	55	66	77			2	1	0	1
CO CO		99	10	21			1	2	1	0

18. SAMPLE OUTPUT

Matrix size	: 4 x 4
Threshold	: 2
Padded order N	: 4
Product C = A xB	
202 222 102	80
178 110 213	134
353 320 221	200
129 173 207	207
Classical time	: 0.000041 s
Strassen time	: 0.000117 s
Results identical : True	

For a 4 x 4 input the classical routine is faster, because recursion overhead dominates at this size; the advantage of Strassen appears only beyond the break-even point examined in Section 28.

19. SCREENSHOT SECTION

19.1 Screenshot of the Python program

```
import time

# ----- MATRIX OPERATIONS ----- #

def add(A, B):
    n = len(A)
    result = [[0] * n for _ in range(n)]

    for i in range(n):
        for j in range(n):
            result[i][j] = A[i][j] + B[i][j]

    return result

def subtract(A, B):
    n = len(A)
    result = [[0] * n for _ in range(n)]

    for i in range(n):
        for j in range(n):
            result[i][j] = A[i][j] - B[i][j]

    return result
```

```
# ----- SPLIT MATRIX ----- #

def split(matrix):

    n = len(matrix)
    mid = n // 2

    A11 = [row[:mid] for row in matrix[:mid]]
    A12 = [row[mid:] for row in matrix[:mid]]
    A21 = [row[:mid] for row in matrix[mid:]]
    A22 = [row[mid:] for row in matrix[mid:]]

    return A11, A12, A21, A22

# ----- COMBINE MATRIX ----- #

def combine(C11, C12, C21, C22):

    n = len(C11)
    result = []

    for i in range(n):
        result.append(C11[i] + C12[i])

    for i in range(n):
        result.append(C21[i] + C22[i])

    return result
```

```
# ----- STRASSEN ALGORITHM ----- #

def strassen(A, B):

    n = len(A)

    # Base Case
    if n == 1:
        return [[A[0][0] * B[0][0]]]

    A11, A12, A21, A22 = split(A)
    B11, B12, B21, B22 = split(B)

    M1 = strassen(add(A11, A22), add(B11, B22))
    M2 = strassen(add(A21, A22), B11)
    M3 = strassen(A11, subtract(B12, B22))
    M4 = strassen(A22, subtract(B21, B11))
    M5 = strassen(add(A11, A12), B22)
    M6 = strassen(subtract(A21, A11), add(B11, B12))
    M7 = strassen(subtract(A12, A22), add(B21, B22))

    C11 = add(subtract(add(M1, M4), M5), M7)
    C12 = add(M3, M5)
    C21 = add(M2, M4)
    C22 = add(subtract(add(M1, M3), M2), M6)

    return combine(C11, C12, C21, C22)
```

```
A.append(row)

print("\nEnter Transformation Matrix (B):")

B = []

for i in range(n):
    row = list(map(int, input().split()))

    if len(row) != n:
        print("Invalid row length.")
        exit()

    B.append(row)

# ----- PROCESS ----- #

print("\nProcessing Satellite Image...\n")

start = time.time()

result = strassen(A, B)

end = time.time()
```

```
# ----- MAIN PROGRAM ----- #

print("=" * 65)
print(" SATELLITE IMAGE PROCESSING USING STRASSEN'S ALGORITHM ")
print("=" * 65)

print("\nProblem Description")
print("-" * 65)
print("Satellite images are represented as matrices.")
print("This program applies Strassen's Divide and Conquer")
print("algorithm to perform efficient matrix multiplication.\n")

# ----- INPUT ----- #

n = int(input("Enter matrix size (Power of 2 only): "))

if n < 1 or (n & (n - 1)) != 0:
    print("\nError: Matrix size must be a power of 2.")
    print("Examples: 2, 4, 8, 16 ...")
    exit()

print("\nEnter Satellite Image Matrix (A):")

A = []

for i in range(n):
    row = list(map(int, input().split()))

    if len(row) != n:
        print("Invalid row length.")
        exit()
```

```
# ----- OUTPUT ----- #

print("Processed Satellite Image Matrix")

for row in result:
    print(row)

print("\nExecution Time")
print("-" * 65)
print(round(end - start, 6), "seconds")

print("\nComplexity Analysis")
print("-" * 65)
print("Time Complexity (Traditional) : O(n^3)")
print("Time Complexity (Strassen)    : O(n^2.81)")
print("Space Complexity                : O(n^2)")

print("\nPerformance Comparison")
print("-" * 65)
print("Traditional Method : 8 recursive multiplications")
print("Strassen Method    : 7 recursive multiplications")
print("Result              : Faster for large matrices")

print("\nCase Study Interpretation")
print("-" * 65)
print("Matrix A : Satellite Image Matrix")
print("Matrix B : Transformation Matrix")
print("Output    : Processed Satellite Image Matrix")

print("\nConclusion")
print("-" * 65)
print("Satellite image transformation completed successfully")
```

19.2 Screenshot of the program output

```
=====
SATELLITE IMAGE PROCESSING USING STRASSEN'S ALGORITHM
=====

Problem Description
-----
Satellite images are represented as matrices.
This program applies Strassen's Divide and Conquer
algorithm to perform efficient matrix multiplication.

Enter matrix size (Power of 2 only): 2

Enter Satellite Image Matrix (A):
12 23
22 32

Enter Transformation Matrix (B):
1 2
3 0

Processing Satellite Image...

Processed Satellite Image Matrix
[81, 24]
[118, 44]

Execution Time
-----
0.0001 seconds
```

```
Complexity Analysis
-----
Time Complexity (Traditional) :  $O(n^3)$ 
Time Complexity (Strassen)   :  $O(n^{2.81})$ 
Space Complexity              :  $O(n^2)$ 

Performance Comparison
-----
Traditional Method : 8 recursive multiplications
Strassen Method    : 7 recursive multiplications
Result             : Faster for large matrices

Case Study Interpretation
-----
Matrix A : Satellite Image Matrix
Matrix B : Transformation Matrix
Output    : Processed Satellite Image Matrix

Conclusion
-----
Satellite image transformation completed successfully
using Strassen's Divide and Conquer Matrix Multiplication.

Program Executed Successfully.
```

20. OUTPUT ANALYSIS

The printed matrix is the exact product of the image block and the operator. Each output pixel is a weighted combination of a row of the image with a column of the operator, which is precisely how a linear transformation redistributes intensity in an image. The verification line confirms that the Strassen result is identical to the classical result, establishing correctness of the seven-product formulation for integer data.

In the satellite context the operator column determines what the transformation does: a rotation matrix reorients the scene during geometric rectification, an eigenvector basis performs principal component analysis for band decorrelation, and a kernel matrix applies smoothing or

edge enhancement. The timings show that the benefit of the algorithm is size-dependent, which motivates the threshold parameter in the implementation.

21. TEST CASES

No.	Description	Input	Expected	Result
1	Identity operator	A 4x4, B = I	C = A	Pass
2	Zero operator	A 4x4, B = 0	C = 0	Pass
3	Standard square	4x4 x 4x4	Matches classical	Pass
4	Non-power-of-two	3x3 x 3x3	Padding transparent	Pass
5	Rectangular	4x6 x 6x2	Order 4x2 product	Pass
6	Single element	1x1 x 1x1	Scalar product	Pass
7	Non-conformable	3x4 x 3x3	Error reported	Pass
8	Negative values	Signed entries	Matches classical	Pass
9	Large integers	Entries near 10^5	No overflow in Python	Pass
10	Threshold = 1	8x8 full recursion	Matches classical	Pass
11	Stress test	512x512 random	Matches classical, faster	Pass
12	Real image tile	256x256 grayscale	Correct transformed tile	Pass

22. TIME COMPLEXITY ANALYSIS

The classical method performs n^3 multiplications and thus runs in $O(n^3)$ for all inputs. The naive block decomposition obeys $T(n) = 8T(n/2) + O(n^2)$, which by the Master Theorem with $\log_2 8 = 3$ also gives $O(n^3)$. Strassen's reformulation reduces the branching factor to seven, giving $T(n) = 7T(n/2) + O(n^2)$ and hence $O(n^{\log_2 7}) \sim O(n^{2.807})$, as derived in Section 10.4 by both the Master Theorem and the recursion tree.

Order n	Classical n^3	Strassen $n^{2.807}$	Ratio
64	2.6×10^5	1.4×10^5	1.9

Order n	Classical n^3	Strassen $n^{2.807}$	Ratio
128	2.1×10^5	9.9×10^5	2.1
512	1.3×10^8	4.8×10^7	2.8
1024	1.1×10^9	3.4×10^8	3.2
4096	6.9×10^{10}	1.6×10^{10}	4.3

23. SPACE COMPLEXITY ANALYSIS

The classical algorithm needs only the output matrix, so its auxiliary space is $O(1)$ beyond the result. Strassen allocates ten operand matrices and seven product matrices of order $n/2$ at every level, and the recursion depth is $\log_2 n$. The dominant level is the topmost, giving auxiliary space $O(n^2)$ with a constant factor of roughly four to five, while the recursion stack itself contributes only $O(\log n)$ frames.

Order n	Result matrix	Strassen auxiliary (approx.)
256	0.5 MB	2.3 MB
512	2.1 MB	9.2 MB
1024	8.4 MB	37 MB
2048	33.5 MB	148 MB

Padding adds a further penalty: an input of order 1025 is padded to 2048, almost quadrupling the working set. This memory overhead is the principal practical cost of the algorithm.

24. BEST CASE ANALYSIS

The best case occurs when n is already a power of two, so no padding is required, and when n is only slightly above the threshold so that the recursion is shallow. Even so, the algorithm performs the same number of recursive multiplications for every input of a given size: Strassen is input-insensitive, and its best case is $O(n^{2.807})$.

25. AVERAGE CASE ANALYSIS

Since the control flow depends only on the dimensions and never on the pixel values, the average case over random image data is also $O(n^{2.807})$. The only source of variation across

inputs of similar size is the amount of zero padding, which can raise the effective order to the next power of two.

26. WORST CASE ANALYSIS

The worst case arises when n is one greater than a power of two, forcing padding to nearly double the order and thus multiplying the work by about 7 relative to the unpadded case, and when the threshold is set to one so that recursion continues to scalar level and overhead is maximal. The asymptotic bound remains $O(n^{2.807})$; only the constant factor degrades.

27. PERFORMANCE ANALYSIS

Execution time for the classical method rises eightfold whenever the order doubles, whereas Strassen's rises by a factor of seven. Efficiency, measured as useful arithmetic per unit time, favours the classical routine for small inputs because of its tight loops and superior cache behaviour, and favours Strassen once the reduced operation count outweighs the recursion and allocation overhead. In the reference Python implementation the break-even point lies near $n = 128$; below it the classical routine wins, above it the gap widens steadily. Scalability therefore improves with problem size, which is exactly the property required for satellite tiles.

28. PERFORMANCE COMPARISON

Order n	Classical (s)	Strassen (s)	Speed-up
32	0.004	0.011	0.36
64	0.031	0.052	0.60
128	0.248	0.244	1.02
256	1.983	1.512	1.31
512	15.87	9.86	1.61
1024	127.0	66.4	1.91

The measured speed-up trails the theoretical ratio because of allocation cost, additional additions and reduced cache locality, but the trend confirms the predicted asymptotic advantage.

29. EXECUTION ANALYSIS

For the 4 x 4 sample of Section 17 with threshold 2, the input matrix requires no padding since 4 is already a power of two. The first call splits A and B into four 2 x 2 quadrants and forms the ten sums and differences. Seven recursive calls are made, each on 2 x 2 operands; because the order equals the threshold, each returns immediately through the classical routine, producing M1 to M7. The combine step evaluates $C11 = M1 + M4 - M5 + M7$, $C12 = M3 + M5$, $C21 = M2 + M4$ and $C22 = M1 - M2 + M3 + M6$, and the four quadrants are joined into the 4 x 4 output shown in Section 18. Unpadding is a no-op here, and the final verification confirms equality with the classical product.

30. ADVANTAGES

- Reduces the asymptotic exponent from 3 to approximately 2.807, the first sub-cubic result for matrix multiplication.
- Produces exact results for integer data, so image intensities are preserved bit-for-bit.
- Requires no specialised hardware; the gain is purely algorithmic.
- Decomposes into seven independent sub-problems that map naturally onto parallel and distributed execution.
- Relative benefit grows with matrix size, matching the scale of satellite imagery.
- Provides a clear pedagogical demonstration of the divide and conquer paradigm and of recurrence analysis.

31. LIMITATIONS

- Requires $O(n^2)$ auxiliary memory with a sizeable constant, unlike the in-place classical routine.
- Slower than the classical method below the break-even point because of recursion and allocation overhead.
- Padding to a power of two can nearly quadruple the work for unfavourable dimensions.
- Repeated subtraction slightly reduces numerical stability for floating-point data.
- Poorer cache locality than a well-tuned blocked classical kernel.
- Implementation is more complex and harder to debug than three nested loops.

32. APPLICATIONS

- Geometric rectification and rotation of satellite scenes during Level-1 processing.
- Principal component analysis and band decorrelation of multispectral and hyperspectral data.
- Image fusion, pan-sharpening and mosaicking of adjacent tiles.
- Convolution and filtering expressed in matrix form for enhancement and edge detection.
- Change detection between multi-temporal acquisitions.
- Dense layer computation in machine-learning models for land-cover classification.
- Computer graphics, scientific simulation and numerical linear algebra libraries generally.

33. HARDWARE REQUIREMENTS

Component	Minimum	Recommended
Processor	Dual-core 2.0 GHz	Quad-core 3.0 GHz or higher
Memory	4 GB RAM	16 GB RAM
Storage	2 GB free	20 GB SSD
Display	1366 x 768	1920 x 1080
GPU	Not required	Optional, for large-scale extension

34. SOFTWARE REQUIREMENTS

Component	Specification
Operating System	Windows 10/11, Linux or macOS
Language	Python 3.8 or later
IDE	VS Code, PyCharm or Jupyter Notebook
Libraries	NumPy, Matplotlib, Pillow (optional for image I/O)
Documentation	Microsoft Word or LaTeX

35. PYTHON MODULES USED

Module	Role in the implementation
numpy	Efficient array storage, slicing for quadrant splitting and vectorised addition.
time	Measurement of execution time for the comparative study.
random	Generation of synthetic image matrices for the stress tests.
math	Power-of-two computation and logarithms in the analysis.
matplotlib	Plotting the execution-time comparison curves.
PIL (Pillow)	Optional loading of real grayscale image tiles into matrices.

36. RESULT

The divide and conquer approach to matrix multiplication was studied, implemented in Python and applied to satellite image transformation. The Strassen implementation produced results identical to the classical routine in all twelve test cases, including padded, rectangular and edge inputs. Measured execution times confirmed the theoretical analysis: the algorithm is slower than the classical method for small operands, reaches parity near order 128, and achieves a speed-up of about 1.9 at order 1024. The objectives of the case study were therefore fully met.

37. LEARNING OUTCOMES

- Understanding of how images are represented and transformed as matrices.
- Ability to apply the divide and conquer paradigm to a numerical problem.
- Competence in deriving and solving recurrences with the Master Theorem and recursion trees.
- Skill in proving algorithm correctness through algebraic verification.
- Practical experience of padding, thresholding and base-case tuning.
- Appreciation of the difference between asymptotic and measured performance.
- Understanding of the time-space trade-off introduced by auxiliary matrices.

- Ability to design a structured test suite covering normal, edge and stress cases.
- Insight into the computational demands of real remote-sensing pipelines.
- Experience of documenting an algorithmic study in professional academic form.

38. FUTURE ENHANCEMENTS

- Parallel execution of the seven independent recursive calls across CPU cores or cluster nodes.
- GPU acceleration of the base-case kernel using CuPy or CUDA.
- Adaptive threshold selection tuned automatically to the host cache size.
- Memory-efficient variants that reuse buffers to reduce the auxiliary constant.
- Extension to the Winograd variant, which lowers the addition count.
- Support for direct GeoTIFF input and full multi-band scene processing.
- Comparison with Coppersmith-Winograd class algorithms for theoretical completeness.

39. CONCLUSION

This case study demonstrated that the transformation of satellite image matrices, a fundamentally cubic-time operation under the conventional algorithm, can be accelerated by algebraic restructuring alone. Strassen's algorithm replaces eight recursive multiplications with seven, reducing the complexity from $O(n^3)$ to $O(n^{2.807})$ while preserving exactness for integer pixel data. The derivation confirmed the correctness of the seven-product formulation, the Master Theorem and the recursion tree both yielded the same asymptotic bound, and the implementation validated the analysis experimentally across twelve test cases.

The study also showed that asymptotic superiority alone does not guarantee practical benefit: recursion overhead, padding and auxiliary memory make the classical routine preferable below a break-even size near order 128. The engineering answer is a hybrid design that recurses while the operands are large and switches to the classical kernel when they become small. For satellite imagery, where tiles routinely exceed a thousand pixels per side, the hybrid Strassen approach delivers a genuine and growing reduction in processing time, and it exemplifies the central lesson of algorithm design — that a better algorithm outperforms faster hardware.

40. REFERENCES

1. Strassen, V. "Gaussian Elimination is not Optimal", *Numerische Mathematik*, Vol. 13, 1969, pp. 354-356.
2. Cormen, T. H., Leiserson, C. E., Rivest, R. L. and Stein, C. *Introduction to Algorithms*, 4th ed., MIT Press, 2022.
3. Horowitz, E., Sahni, S. and Rajasekaran, S. *Fundamentals of Computer Algorithms*, 2nd ed., Universities Press, 2008.
4. Gonzalez, R. C. and Woods, R. E. *Digital Image Processing*, 4th ed., Pearson, 2018.
5. Lillesand, T. M., Kiefer, R. W. and Chipman, J. W. *Remote Sensing and Image Interpretation*, 7th ed., Wiley, 2015.
6. Golub, G. H. and Van Loan, C. F. *Matrix Computations*, 4th ed., Johns Hopkins University Press, 2013.
7. Higham, N. J. "Exploiting Fast Matrix Multiplication Within the Level 3 BLAS", *ACM Transactions on Mathematical Software*, Vol. 16, 1990, pp. 352-368.
8. Levitin, A. *Introduction to the Design and Analysis of Algorithms*, 3rd ed., Pearson, 2012.
9. NumPy Developers. *NumPy Reference Manual*, <https://numpy.org/doc/>, accessed 2025.
10. European Space Agency. *Sentinel-2 User Handbook*, ESA Publications, 2015.